

Foodshare



Készítette:

Tábi Ákos

Soós Nagy Márk

Bevezetés	2
Fejlesztői dokumentáció	3
Fejlesztői környezet.....	4
Kialakított adatszerkezet részletes bemutatása.	4
Users	5
Recipes.....	6
Comments	8
Ratings	9
Tags	10
Tag_Categories	11
Recipe_Tags	11
User_Default_Tags	12
Roles	12
UserRole	13
Algoritmusok.....	13
Recept létrehozása címkékkel együtt	14
Címkék csoportosítása kategóriák szerint	15
Csillagos értékelés kirajzolása kerekített átlag alapján	16
Kategóriák lenyitása és összecsukása tageknél	17
Felhasználó alapértelmezett tagjeinek előtöltése.....	19
Menü generálása véletlenszerű receptválasztással és tagek szerinti szűréssel	19
A menü generálása.....	21
Bejelentkezés kezelése	21
Tesztelés	27
Funkcionális tesztelés	28
Usability tesztelés	28
Felhasználói felületek tesztelése.....	29
Kompatibilitás tesztelés	30
Teljesítmény tesztelés.....	31
Felhasználói dokumentáció.....	31
1. A program bemutatása	32
2. Hardverkövetelmények	32

3. Szoftverkövetelmények	33
4. A program letöltése, telepítése és indítása	34
5. A program használata	35
5.1 Főoldal	36
5.2 Receptek böngészése	36
5.3 Menü	37
5.4 Új recept hozzáadása	38
6. Szabadon használható funkciók és regisztrációhoz kötött funkciók	39
7. Szövegmezők követelményei	40
8. Hibajelzések	41
9. Admin felhasználó lehetőségei	42
11. Kapcsolat és támogatás	43
Összefoglalás	44
A munka értékelése	45
A program jövőbeli hasznosulása	45

Bevezetés

Manapság sokan szeretnének változatosabban étkezni, ugyanakkor a hétköznapi rohanásban gyakran nehéz kitalálni, mit együnk, különösen akkor, ha nemcsak egy napra, hanem akár egy egész hétre előre kell tervezni. Bár már léteznek különböző receptmegosztó oldalak, ezek többnyire csak egy-egy konkrét étel keresésében segítenek, és kevésbé támogatják a tudatosabb, előre átgondolt étkezéstervezést. Az ember gyakran csak böngészik a receptek között, de végül mégsem áll össze egy átlátható terv arra, hogy mit mikor érdemes elkészíteni.

Személyesen is sokszor tapasztalom, hogy kifejezetten nehéz dönteni arról, mit egyek. Előfordul, hogy hosszú percekig, akár negyedórákig is gondolkodom ezen, mégsem jutok igazán jó megoldásra. Ilyenkor vagy ugyanazokat az ételeket választom újra és újra, vagy valami gyors, de kevésbé változatos megoldás mellett döntök. Ez nemcsak időigényes, hanem egy idő után egyhangúvá is teszi az étkezést.

Egy olyan weboldal, ahol a receptek mellett lehetőség van heti menü generálására is, jelentős segítséget nyújthatna ebben. Nemcsak inspirációt adna új ételek kipróbálásához, hanem segítene megszerezni is az ötleteket, és konkrét tervet kínálna a hét minden napjára. Ez megkönnyítené a bevásárlás megtervezését, csökkentené a felesleges döntési helyzeteket, és összességében hatékonyabbá tenné a mindennapi étkezés megszervezését.

Fejlesztői dokumentáció

Fejlesztői környezet

A backendhez és frontendhez is a Visual Studio-t használtuk kódszerkesztőnek, azon belül az Asp.Net Core Web API és Blazor Web App sablonokból kiindulva, közös objektumokat egy ClassLibrary-ban tárolva, mivel a C# kódnyelv használata és ezáltal könnyű kapcsolat közöttük egyszerűvé és könnyen átláthatóvá tette a fejlesztési folyamatot.

A weboldal adatbázisának létrehozásához és kezeléséhez a SQL Server Management Studio adatbáziskezelő programot használtuk hasonló indokokból.

Valamint mindkét programmal való tapasztalatunk és a széles körű elérhetőségük miatt nem csak ezekkel volt a legtöbb tapasztalatunk, de az interneten található rengeteg információnak és segítőkész oktatói anyagnak köszönhetően könnyen lehetett a hibáinkra és problémáinkra megoldást találni.

A weboldalt a Microsoft Azure segítségével futtatjuk az adatbázistól a frontending. Ezt a választást az Azure ingyenes opciói valamint az integrálása a Visual Studio-val indokolta.

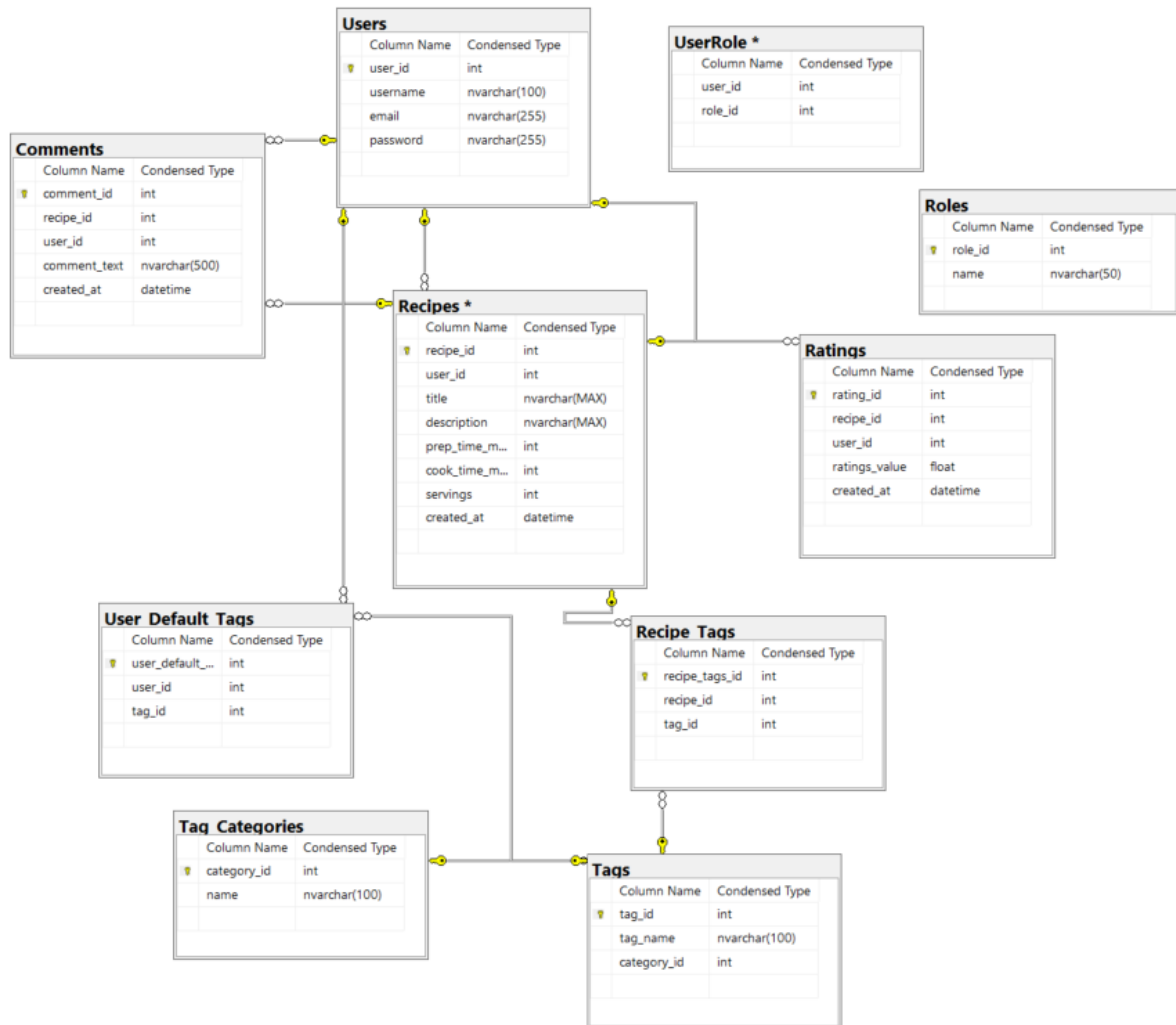
A munkán a Github segítségével osztottunk az alábbi linken:
<https://github.com/Takos05/ReceptMegosztRepo>

Kialakított

adatszerkezet

részletes

bemutatása.



Users

Users		
	Column Name	Condensed Type
🔑	user_id	int
	username	nvarchar(100)
	email	nvarchar(255)
	password	nvarchar(255)

A **user_id** mező a Users tábla elsődleges kulcsa, amely egy automatikusan növekvő egész szám, és minden felhasználót egyértelműen azonosít a rendszerben. Ez a mező kulcsszerepet játszik az adatbázis kapcsolatrendszerében, mivel több másik tábla is erre hivatkozik idegen kulcsként. Például a Recipes, Comments, Ratings, User_Default_Tags és UserRole táblák mind tartalmazzák ezt az értéket, így minden felhasználóhoz köthető tevékenység ezen keresztül kapcsolódik vissza.

A **username** mező a felhasználó megjelenített nevét tartalmazza. Ez az adat, amely a felhasználói felületen is látható, például kommenteknél vagy más interakciók során. Bár közvetlen kapcsolatot nem hoz létre más táblákkal, mégis fontos szerepe van abban, hogy a Users tábla adatai értelmezhetőek legyenek más táblákban, például amikor a Comments tábla user_id mezőjén keresztül egy felhasználóhoz tartozó nevet jelenítünk meg.

A **email** mező a felhasználó egyedi elérhetőségét tárolja. Ez az adat elsősorban azonosításra és kommunikációra szolgál, és bár nem kapcsolódik közvetlenül más táblákhoz idegen kulcson keresztül, a rendszer logikájában fontos szerepet játszik a felhasználók egyediségének biztosításában.

A **password** mező a felhasználó jelszavát tartalmazza. Ez az adat a hitelesítéshez szükséges, és kizárólag a Users táblán belül használatos, más táblákhoz közvetlen kapcsolat nem kötődik hozzá, azonban elengedhetetlen a rendszer biztonságos működéséhez.

Recipes

Recipes *		
	Column Name	Condensed Type
🔑	recipe_id	int
	user_id	int
	title	nvarchar(MAX)
	description	nvarchar(MAX)
	prep_time_m...	int
	cook_time_m...	int
	servings	int
	created_at	datetime

A **recipe_id** mező a Recipes tábla elsődleges kulcsa, amely minden receptet egyedileg azonosít. Ez az azonosító jelenik meg más táblákban is, például a Comments, Ratings és Recipe_Tags táblákban, ahol idegen kulcsként szolgál, és biztosítja, hogy minden kapcsolódó adat egy adott recepthez legyen köthető.

A **user_id** mező azt a felhasználót jelöli, aki a receptet létrehozta, és közvetlen kapcsolatot hoz létre a Users táblával. Ez a kapcsolat lehetővé teszi, hogy minden recepthez hozzárendeljük a készítőjét, valamint azt is, hogy lekérdezzük egy adott felhasználó által létrehozott recepteket.

A **title** mező a recept nevét tartalmazza, amely a felhasználói felületen is megjelenik. Ez az adat nem kapcsolódik más táblákhoz idegen kulcson keresztül, de kulcsfontosságú az adatok értelmezésében és a receptek azonosításában.

A **description** mező a recept teljes leírását tárolja, amely tartalmazhatja a hozzávalókat, az elkészítési lépéseket és egyéb részleteket. Ez a mező nagy méretű szöveges adat tárolására alkalmas, és bár nem kapcsolódik más táblákhoz, a rendszer tartalmi részének legfontosabb eleme.

A **prep_time_min** mező az előkészítési időt tárolja percben. Ez az adat nem hoz létre kapcsolatot más táblákkal, de fontos információ a felhasználók számára a recept kiválasztásakor.

A **cook_time_min** mező a főzési vagy sütési időt tartalmazza percben. Hasonlóan az előkészítési időhöz, ez is egy önálló adat, amely nem kapcsolódik más táblákhoz, de a recept gyakorlati használatához szükséges.

A **servings** mező az adagok számát tárolja. Ez az információ szintén nem kapcsolódik más táblákhoz, de segíti a felhasználókat a recept mennyiségi értelmezésében.

A **created_at** mező a recept létrehozásának időpontját rögzíti. Ez az adat lehetővé teszi az idő szerinti rendezést és a receptek időbeli nyomon követését, bár közvetlen kapcsolatot nem hoz létre más táblákkal.

Comments

Comments		
	Column Name	Condensed Type
	comment_id	int
	recipe_id	int
	user_id	int
	comment_text	nvarchar(500)
	created_at	datetime

A **comment_id** mező a Comments tábla elsődleges kulcsa, amely minden hozzászólást egyedileg azonosít. Ez a mező biztosítja, hogy minden komment külön kezelhető legyen az adatbázisban.

A **recipe_id** mező idegen kulcsként kapcsolódik a Recipes tábla recipe_id mezőjéhez. Ez határozza meg, hogy az adott komment melyik recepthoz tartozik, így lehetővé teszi a kommentek recept szerinti lekérdezését.

A **user_id** mező idegen kulcsként kapcsolódik a Users tábla user_id mezőjéhez. Ez mutatja meg, hogy ki írta a kommentet, és lehetővé teszi a felhasználók és hozzászólásaik közötti kapcsolat kezelését.

A **comment_text** mező a hozzászólás szövegét tartalmazza. Ez az adat nem kapcsolódik más táblákhoz, de a rendszer interaktív működésének alapját képezi.

A **created_at** mező a komment létrehozásának időpontját tárolja, amely lehetővé teszi a kommentek időrendi rendezését és megjelenítését.

Ratings

Ratings		
	Column Name	Condensed Type
🔑	rating_id	int
	recipe_id	int
	user_id	int
	ratings_value	float
	created_at	datetime

A **rating_id** mező a Ratings tábla elsődleges kulcsa, amely minden értékelést egyedileg azonosít.

A **recipe_id** mező idegen kulcsként kapcsolódik a Recipes táblához, és azt jelzi, hogy melyik recepthez tartozik az értékelés.

A **user_id** mező idegen kulcsként kapcsolódik a Users táblához, és azt mutatja meg, hogy melyik felhasználó adta az értékelést.

A **ratings_value** mező az értékelés numerikus értékét tartalmazza, amely alapján átlag számítható egy adott receptre.

A **created_at** mező az értékelés időpontját rögzíti, ami lehetővé teszi az értékelések időbeli elemzését.

Tags

Tags			
	Column Name	Condensed Type	
🔑	tag_id	int	
	tag_name	nvarchar(100)	
	category_id	int	

A **tag_id** mező a Tags tábla elsődleges kulcsa, amely minden címkét egyedileg azonosít.

A **tag_name** mező a címke nevét tartalmazza, amely leírja a recept egy jellemzőjét.

A **category_id** mező idegen kulcsként kapcsolódik a Tag_Categories tábla category_id mezőjéhez, így minden címke egy kategóriához rendelhető.

Tag_Categories

Tag_Categories			
	Column Name	Condensed Type	
🔑	category_id	int	
	name	nvarchar(100)	

A **category_id** mező a Tag_Categories tábla elsődleges kulcsa, amely minden kategóriát egyedileg azonosít.

A **name** mező a kategória megnevezését tartalmazza, amely alapján a címkék csoportosíthatók.

Recipe_Tags

Recipe Tags		
	Column Name	Condensed Type
🔑	recipe_tags_id	int
	recipe_id	int
	tag_id	int

A **recipe_tags_id** mező a kapcsolótábla elsődleges kulcsa, amely minden kapcsolatot egyedileg azonosít.

A **recipe_id** mező idegen kulcsként kapcsolódik a Recipes táblához, és azt jelzi, hogy melyik recepthez tartozik a kapcsolat.

A **tag_id** mező idegen kulcsként kapcsolódik a Tags táblához, és azt jelzi, hogy melyik címke van hozzárendelve az adott recepthez.

User_Default_Tags

User Default Tags		
	Column Name	Condensed Type
🔑	user_default_...	int
	user_id	int
	tag_id	int

A **user_default_tags_id** mező a kapcsolótábla elsődleges kulcsa, amely minden rekordot egyedileg azonosít.

A **user_id** mező idegen kulcsként kapcsolódik a Users táblához, és azt jelzi, hogy melyik felhasználóhoz tartozik a címke.

A **tag_id** mező idegen kulcsként kapcsolódik a Tags táblához, és azt jelzi, hogy melyik címke van hozzárendelve a felhasználóhoz.

Roles

Roles		
	Column Name	Condensed Type
🔑	role_id	int
	name	nvarchar(50)

A **role_id** mező a Roles tábla elsődleges kulcsa, amely minden szerepkört egyedileg azonosít.

A **name** mező a szerepkör megnevezését tartalmazza, amely alapján a jogosultságok értelmezhetők.

UserRole

UserRole *		
	Column Name	Condensed Type
	user_id	int
	role_id	int

A **user_id** mező idegen kulcsként kapcsolódik a Users táblához, és azt jelzi, hogy melyik felhasználóhoz tartozik a kapcsolat.

A **role_id** mező idegen kulcsként kapcsolódik a Roles táblához, és azt jelzi, hogy melyik szerepkör van hozzárendelve a felhasználóhoz.

Algoritmusok

Recept létrehozása címkékkel együtt

A recept létrehozása a backendben nem egyetlen egyszerű mentési művelet, hanem két egymásra épülő részfolyamatból áll. Először maga a recept jön létre a Recipes táblában, majd csak ezután kerülnek rögzítésre a hozzá tartozó címkekapcsolatok a Recipe_Tags táblában. Ez a logika a RecipeController CreateRecipeWithTags metódusában található.

A folyamat menete

1. **Kérés ellenőrzése**
 - a. A metódus első lépésben megvizsgálja, hogy a kapott request objektum létezik-e.
 - b. Ha a kérés null, a rendszer BadRequest hibát ad vissza az „Érvénytelen kérés.” üzenettel.
 - c. Ezzel megakadályozza, hogy hiányos vagy teljesen üres adatszerkezeten fusson le a további logika.
2. **Új receptobjektum összeállítása**
 - a. A rendszer létrehoz egy új Recipes példányt.
 - b. A mezők a kérésből kerülnek átmásolásra:
 - i. user_id
 - ii. title
 - iii. description
 - iv. prep_time_min
 - v. cook_time_min
 - vi. servings
 - c. A created_at mezőt a backend állítja be DateTime.Now értékkel, tehát a létrehozás időpontját nem a kliens küldi.
3. **A recept mentése**
 - a. A rendszer hozzáadja az új receptet a Recipes halmazhoz az _context.Recipes.Add(recipe) hívással.
 - b. Ezután SaveChangesAsync() fut le.
 - c. Erre azért van szükség, mert a recept elsődleges kulcsa, a recipe_id, csak az adatbázisba mentés után áll rendelkezésre.
4. **Címkelista feldolgozása**
 - a. A rendszer ellenőrzi, hogy a tag_ids lista létezik-e, és tartalmaz-e elemet.
 - b. Ha igen, akkor a lista elemein lefuttat egy Distinct() műveletet.
 - c. Ez eltávolítja az ismétlődő címkeazonosítókat, tehát ha ugyanaz a címke többször érkezik meg, a kapcsolat csak egyszer jön létre.
5. **Kapcsolórekordok létrehozása**

- a. A címkeazonosítókból a rendszer `Select(...)` segítségével `Recipe_Tags` objektumokat készít.
 - b. Minden rekord két kulcsértéket kap:
 - i. `recipe_id = recipe.recipe_id`
 - ii. `tag_id = tagId`
 - c. Az így létrejött lista lesz a recept és a címkék közötti kapcsolatok halmaza.
6. **Kapcsolatok mentése**
- a. A backend az összes új `Recipe_Tags` rekordot egyszerre adja hozzá az adatbázis-kontextushoz.
 - b. Ezután újra `SaveChangesAsync()` fut le.

Címkék csoportosítása kategóriák szerint

A `TagsController GetTagsByCategories` metódusa az egyik legösszetettebb címke-kezelő algoritmus. Itt a cél már nem egyszerű lista visszaadása, hanem csoportosított adatszerkezet létrehozása.

A folyamat menete

1. **Címkék csoportosítása**
 - a. A rendszer a tags halmazon `GroupBy(t => t.category_id)` műveletet hajt végre.
 - b. Ennek eredményeként minden csoport egy adott kategóriaazonosítóhoz tartozó címkéket tartalmaz.
2. **Kategóriák betöltése**
 - a. A backend a `tag_Categories` táblát is teljes egészében beolvassa.
3. **Eredménytároló létrehozása**
 - a. A rendszer létrehoz egy `Dictionary<string, string[]>` típusú változót.
 - b. Ebben a kulcs a kategória neve, az érték pedig az adott kategóriához tartozó címkenevek tömbje lesz.
4. **Csoportok bejárása**
 - a. A metódus `foreach` ciklusban végigmegy minden címkecsoporton.
5. **Kategórianév feloldása**
 - a. Minden csoportnál a backend megkeresi a hozzá tartozó kategóriát:
 - i. `categories.FirstOrDefault(c => c.category_id == category.Key)`
 - b. Így az azonosítóból név lesz.
6. **Ideiglenes tömb létrehozása**
 - a. A rendszer létrehoz egy `string[]` temp tömböt, amelynek mérete megegyezik a csoport elemszámával.
7. **Címkenevek bemásolása**
 - a. Egy belső ciklus végigmegy az adott kategóriába tartozó címkéken.
 - b. A `tag.tag_name` értékek sorban bekerülnek a temp tömbbe.
8. **Kulcs-érték pár hozzáadása**
 - a. Az elkészült tömb a kategórianévhez kapcsolódva bekerül a `result` szótárba.
9. **Válasz visszaadása**
 - a. A rendszer a teljes szótárat küldi vissza.

Csillagos értékelés kirajzolása kerekített átlag alapján

A ReceptekDetails.razor oldalon az átlagos értékelés nemcsak számmal jelenik meg, hanem csillagokkal is. Ehhez külön algoritmus alakítja át az átlagot megjeleníthető csillagszámmá.

A működés menete

1. A rendszer lekéri az átlagos értékelést:
 - a. `/api/Rating/averageByRecipe?recipeId={id}`
2. A kapott lebegőpontos értéket eltárolja az `averageRating` változóban.
3. Ezután meghívja:
 - a. `Math.Round(averageRating, MidpointRounding.AwayFromZero)`
4. Az eredmény egész számként a `roundedStars` változóba kerül.
5. Ezután egy korrekciós lépés fut:
 - a. ha az érték kisebb mint 0, akkor 0 lesz
 - b. ha az érték nagyobb mint 5, akkor 5 lesz
6. A megjelenítés egy for ciklusban történik 1-től 5-ig:
 - a. ha az aktuális ciklusérték kisebb vagy egyenlő a `roundedStars` értékével, telt csillag jelenik meg
 - b. különben üres csillag jelenik meg

Például:

- 4.2 átlag → 4 csillag
- 4.5 átlag → 5 csillag
- 2.6 átlag → 3 csillag

Kategóriák lenyitása és összezsukása tageknél

A RecipeCreate.razor, RecipeEdit.razor és a RecipeMenu oldalon a tagek kategóriák szerint vannak csoportosítva, és minden kategória külön nyitható vagy csukható. Ezt egy állapotkezelési algoritmus vezérli, amely egy közös adatszerkezetben tárolja a kategóriák aktuális állapotát.

Használt adatszerkezet

A komponens egy

`HashSet<string> collapsedCategories`

halmazt használ.

Ez a halmaz azoknak a kategóriáknak a nevét tartalmazza, amelyek **összezsukott állapotban vannak**.

- ha egy kategória neve benne van → csukott
- ha nincs benne → nyitott

A rendszer tehát csak a zárt kategóriákat tárolja, a nyitottakat nem.

A működés menete

1. A felhasználó rákattint egy kategória fejlécére
2. Meghívódik a `ToggleCategory(category.Key)` metódus
3. A metódus ellenőrzi, hogy a kategórianév szerepel-e a `collapsedCategories` halmazban (`Contains`)

Ezután:

- **ha benne van:**
 - o eltávolítja (`Remove`)
 - o → a kategória kinyílik
- **ha nincs benne:**
 - o hozzáadja (`Add`)
 - o → a kategória összezsukódik

Ez minden kattintásnál az adott kategória állapotának megfordítását jelenti.

Az állapot ellenőrzése

A megjelenítés során a rendszer a

`IsCategoryCollapsed(category.Key)`

függvényt használja, amely lényegében ezt vizsgálja:

- `collapsedCategories.Contains(category.Key)`

Az eredmény alapján:

- `true` → a kategória csukott
- `false` → a kategória nyitott

Felhasználó alapértelmezett tagjeinek előtöltése

A `Profile.razor` és a `Menu.razor` oldalakon az alapértelmezett tagek szekció nem üresen jelenik meg, hanem a korábban elmentett felhasználói beállításokkal töltődik fel.

A működés menete

1. A rendszer betölti az összes elérhető tagot.
2. Betölti a kategorizált taglistát.
3. Ha az aktuális felhasználó azonosítója ismert, külön API-hívást indít:
 - a. `api/tags/UserDefaults?userId={currentUserId}`
4. A válaszként kapott taglistából kiválasztja a `tag_id` mezőket.
5. Ezeket `ToHashSet()` segítségével halmazzá alakítja.
6. Az eredmény bekerül a `selectedTagIds` változóba.
7. A checkboxok kirajzolásakor a rendszer ezt nézi:
 - a. ha egy tag azonosítója benne van a halmazban, akkor a checkbox pipált lesz

Menü generálása véletlenszerű receptválasztással és tagek szerinti szűréssel

A Menu.razor oldalon a menü elkészítése két egymásra épülő lépésből áll. Először a rendszer leszűri a használható recepteket a kiválasztott tagek alapján, majd ebből a szűrt halmazból véletlenszerűen kiválasztja a menübe kerülő elemeket.

Használt adatok

A folyamat során több lista is szerepet kap:

- az összes recept listája
- a kiválasztott tagek listája
- a szűrt receptek listája (filteredRecipes)
- a végső menü (menu)

A működés menete

1. A menuCreate() metódus elején a rendszer kiüríti a korábbi menüt:
 - a. menu.Clear()
2. Ezután eldönti, hogy szükséges-e szűrés:
 - **ha nincs kiválasztott tag:**
 - a. az összes recept használható
 - **ha vannak kiválasztott tagek:**
 - a. meghívódik a FilterRecipesByTags() metódus
 - a. ez visszaad egy szűrt receptlistát

A szűrés működése

A FilterRecipesByTags() metódus a következő lépéseket végzi:

1. Létrehoz egy üres listát:
 - a. filteredRecipes
2. Végigmegy az összes recepten.
3. Minden receptnél külön lekéri a hozzá tartozó tageket:
 - a. api/Tags/byRecipe/{recipeId}
4. A kapott tagokból kinyeri a tageveket.
5. Megvizsgálja, hogy a kiválasztott tagek közül van-e olyan, amely szerepel a recept tagjai között.
6. Ha van egyezés:
 - a. a recept bekerül a filteredRecipes listába

7. A folyamat végén visszaadja a szűrt listát.

A menü generálása

Miután a rendszer rendelkezik egy használható receptlistával (szűrt vagy teljes), megkezdődik a véletlenszerű kiválasztás.

1. Ha a lista üres, vagy nincs kiválasztott nap:
 - a. a folyamat leáll
2. A rendszer létrehoz egy véletlenszám-generátort:
 - a. Random
3. Meghatározza, hány receptet kell kiválasztani:
 - a. a kiválasztott napok száma
 - b. és a rendelkezésre álló receptek száma közül a kisebbik
4. Ezután ciklus indul.
5. Minden lépésben:
 - a. kiválaszt egy véletlen indexet
 - b. kiválasztja a hozzá tartozó receptet
6. A kiválasztott recept:
 - a. bekerül a menu listába
7. Ezután a rendszer eltávolítja ezt a receptet a forráslistából.

Bejelentkezés kezelése

A projektben a bejelentkezés két részből áll: egy **frontend oldali űrlapkezelésből** és egy **backend oldali hitelesítési folyamatból**. A felhasználó a profiloldalon adja meg az adatait, a rendszer ezeket elküldi az API-nak, majd sikeres belépés esetén a kapott tokenből felépít egy bejelentkezett felhasználói állapotot.

A használt adatok

A bejelentkezési folyamatban a következő mezők szerepelnek:

- Username
- Password

A frontend oldalon ezeket a rendszer a

loginModel

objektumban kezeli.

A bejelentkezési űrlap a Profile.razor oldalon jelenik meg, ahol a felhasználó két mezőt tölt ki:

- felhasználónév
- jelszó

Az űrlap nem közvetlenül az API-t hívja meg, hanem ezt az útvonalat használja:

```
<form method="post" action="/auth/login-form">
```

Ez azt jelenti, hogy a kérés először a frontend saját FrontAuthController vezérlőjéhez érkezik.

A működés menete a frontend oldalon

1. A felhasználó megadja a felhasználónevét és jelszavát a profiloldalon.
2. Az űrlap elküldésekor a böngésző POST kérést küld a következő címre:

/auth/login-form

1. A kérésben a következő adatok szerepelnek:
 - a. Username
 - b. Password
 - c. returnUrl
2. A FrontAuthController LoginForm metódusa fogadja ezt a kérést.
3. Ez a metódus nem helyben ellenőrzi a felhasználót, hanem egy új kérésobjektumot készít az API számára:

```
var payload = new
{
    username = model.Username,
    email = "",
    password = model.Password
};
```

Itt látható, hogy a backend felé a rendszer három mezőt küld:

- username
- email
- password

Az email itt üres szöveggént szerepel, mert a bejelentkezés ténylegesen felhasználónév és jelszó alapján történik.

6. Ezután a frontend vezérlő elküldi a kérést az API-nak:
api/auth/login

A működés menete a backend oldalon

A backend oldalon a ReceptekApi projekt AuthController vezérlője kezeli a bejelentkezést.

A Login(Users login) metódus egy Users objektumot kap, amelyben a következő mezők fontosak:

- username
- password

A működés lépései:

1. A rendszer a beérkezett jelszót nem közvetlenül hasonlítja össze az adatbázisban tárolt értékkel, hanem először lefuttat rajta egy hash-elést.
2. Ehhez a következő logika tartozik:
 - a. SHA256 algoritmust használ
 - b. a jelszót UTF-8 formában byte-sorrá alakítja
 - c. az eredményből Base64 szöveget készít
3. Az így kapott hash kerül összehasonlításra az adatbázisban tárolt jelszóval.
4. Ezután a rendszer megkeresi azt a felhasználót, akire egyszerre igaz, hogy:
 - a. a username megegyezik
 - b. a password hash megegyezik
5. Ha nincs ilyen rekord:
 - a. a rendszer Unauthorized("Sikertelen bejelentkezés") választ ad vissza
6. Ha a felhasználó létezik, akkor a backend még lekéri a felhasználó szerepköreit is.

Ez a szerepkör-lekérés több táblán keresztül történik:

- userRole
- roles

A lekérdezés végeredménye egy névlista, például:

- User
- vagy esetleg Admin

A token előállítás

Ha a felhasználónév és jelszó helyes, a rendszer meghívja a JWT token-generálást.

A tokenbe a következő fontos adatok kerülnek bele:

- ClaimTypes.Name → a felhasználónév
- ClaimTypes.NameIdentifier → a felhasználó azonosítója
- sub → a felhasználó azonosítója
- unique_name → a felhasználónév
- jti → egy egyedi tokenazonosító
- szerepkörök → például User vagy Admin

Ezután a backend egy ilyen szerkezetű választ küld vissza:

```
return Ok(new
{
    token = token
});
```

Tehát a frontend valójában egy JWT token-t kap vissza.

A token feldolgozása a frontend oldalon

Miután a FrontAuthController megkapja az API válaszát, a következő történik:

1. A válaszból kiolvassa a token mezőt.
2. Ellenőrzi, hogy a token valóban létezik-e.
 - a. ha hiányzik, a bejelentkezés hibával leáll
3. Ha van token, a rendszer ezt nem egyszerű szöveggént tárolja el, hanem feldolgozza:

```
var handler = new JwtSecurityTokenHandler();
var jwt = handler.ReadJwtToken(loginResult.token);
```

4. A tokenből kiolvassa a claim-eket:

```
var claims = new List<Claim>(jwt.Claims);
```

5. Ezután külön ellenőrzi, hogy van-e már ClaimTypes.Name claim.
6. Ha nincs, akkor a rendszer megpróbálja pótolni a tokenből kapott más névből, például:
 - unique_name
 - name

Ez azért fontos, mert a frontend több helyen a User.Identity?.Name értéket használja, ezért biztosítani kell, hogy ez a claim mindig rendelkezésre álljon.

A bejelentkezett állapot létrehozása

A frontend nem a JWT token használja közvetlenül minden oldalon, hanem cookie-alapú bejelentkezett állapotot hoz létre.

A folyamat lépései:

1. A claim-ekből létrejön egy ClaimsIdentity
2. Ennek hitelesítési típusa:

`CookieAuthenticationDefaults.AuthenticationScheme`

3. Ezután a rendszer létrehoz egy ClaimsPrincipal objektumot
4. Végül a felhasználót bejelentkezteti:

```
await HttpContext.SignInAsync(  
    CookieAuthenticationDefaults.AuthenticationScheme,  
    principal);
```

Ez azt jelenti, hogy a frontend a sikeres API-hitelesítés után **cookie-s bejelentkezést** hoz létre a saját oldalán.

Mi történik sikeres bejelentkezés után

Ha minden lépés sikeres volt, a rendszer átirányítja a felhasználót:

```
return LocalRedirect(string.IsNullOrEmpty(returnUrl) ? "/" : returnUrl);
```

A Profile.razor oldalon a returnUrl értéke:

```
/profile
```

Tehát sikeres belépés után a felhasználó visszakerül a profiloldalra, de már bejelentkezett állapotban.

Ekkor a komponens újra lekéri az AuthenticationStateProvider állapotát, és a következő adatokhoz fér hozzá:

- felhasználónév
- email
- felhasználóazonosító
- szerepkörök

A bejelentkezett adatok kiolvasása

A profiloldalon a rendszer az AuthenticationStateProvider segítségével olvassa ki az aktuális felhasználó adatait.

A fontosabb claim-ek, amelyeket keres:

- ClaimTypes.Name
- ClaimTypes.Email
- ClaimTypes.NameIdentifier

A NameIdentifier alapján a rendszer meghatározza a felhasználó user_id értékét, és ezt később több helyen is felhasználja, például:

- saját receptek lekérése
- alapértelmezett tagek betöltése
- kommentek és értékelések kezelése

Sikertelen bejelentkezés kezelése

Ha az API hibát ad vissza, a FrontAuthController nem hoz létre bejelentkezett állapotot.

A LoginForm esetén ilyenkor ez történik:

```
return Redirect($"{returnUrl}?error=1");
```

A profiloldal induláskor ellenőrzi, hogy az URL-ben szerepel-e az error=1 paraméter. Ha igen, akkor:

- showLoginError = true

és a felületen megjelenik a hibaüzenet:

- „Sikertelen bejelentkezés.”

Tesztelés

A webes alkalmazásunk tesztelésénél az alábbi módszertani lépéseket követtük:

- Funkcionális tesztelés
- Usability tesztelés
- Felhasználói felületek tesztelése
- Kompatibilitás tesztelés
- Teljesítmény tesztelés

Funkcionális tesztelés

Első lépésként funkcionális tesztelést csináltunk, melynek célja annak ellenőrzése volt, hogy az alkalmazás minden részegysége a specifikációnk szerint működik-e. A funkcionális tesztelésnél megnéztük például az alábbiakat:

sikeres regisztrációról érkezik-e visszajelzés a felületen

a profil oldalon beállított alapértelmezett értékek megjelennek-e a menü oldalon

a menü létrehozás funkció a kijelölt napokra és a bejelölt tag-enknek megfelelő receptek közül választva ajánl-e megfelelő ételeket.

A funkcionális tesztelés része volt az űrlap, jelen esetben a regisztrációs felület tesztelése kiemelt figyelemmel a hiba üzenetekre és a mező validációra, valamint az új recept feltöltés funkció, mint űrlap, ahol ellenőriztük többek között a kötelezően kitöltendő mezők kezelését. A belépés funkcionál a jelszó maszkolás megfelelő működését kellett tesztelni.

A funkcionális teszteléshez tartozott természetesen annak ellenőrzése, hogy minden kattintható elem elvezet-e a megfelelő oldalra: a linkek ellenőrzésénél nem csak a hibás linkeket szűrtük ki, hanem azt is teszteltük, hogy minden link a megfelelő oldalra mutat-e. A funkcionális tesztelés során ellenőriztük, hogy a recept feltöltésnél, illetve az alapértelmezett tag-ek beállításánál az adatbázisba történő mentés megfelelő-e.

Usability tesztelés

Usability tesztelés során azt néztük meg, hogy a weboldal navigációja konzekvens-e, könnyen kezelhető-e, minden link megfelelően működik-e. A tesztelés ezen fázisába bevontuk családunk tagjait is, akik több hasznos észrevételt tettek, elsősorban helyesírási hibák, megfogalmazások tekintetében, melyeket természetesen figyelembe vettünk. Figyeltünk arra, hogy a szövegek kontrasztosak, könnyen olvashatók legyenek. A usability tesztelés része volt a recept kereső funkció működésének ellenőrzése. A családtagok visszajelzése alapján egy helyen kellett módosítást csinálni: a recept feltöltésnél a mentés után nem volt egyértelmű, hogy sikeres volt-e a feltöltés, volt, akinek kétszer is sikerült feltöltenie ugyanazt a receptet. Ezt a problémát oldottuk meg úgy, hogy a mentés után a feltöltött recept jelenik meg, így egyből ellenőrizhető annak tartalma.

Felhasználói felületek tesztelése

A felhasználói felület tesztelése során ellenőrizni kellett a design elemeket: az elrendezést, a használt színeket, betűtípusokat, a szövegformázásokat stb. A weboldal tesztelése során elengedhetetlen volt a reszponzív kialakítás ellenőrzése különböző eszközökön. A tesztelést három fő kategóriára bontottuk: mobil, tablet és asztali számítógép. Asztali számítógép esetén az alábbi felbontásokat teszteltük:

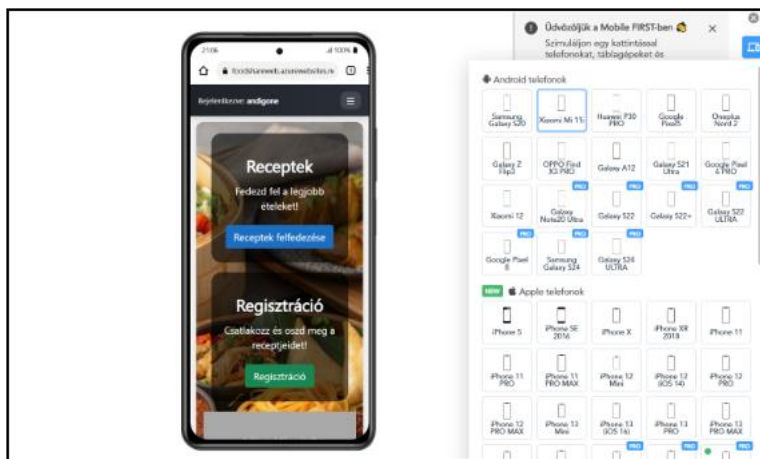
1920x1080 (a legelterjedtebb asztali felbontás)

1366x768 (széles körben használt laptop felbontás)

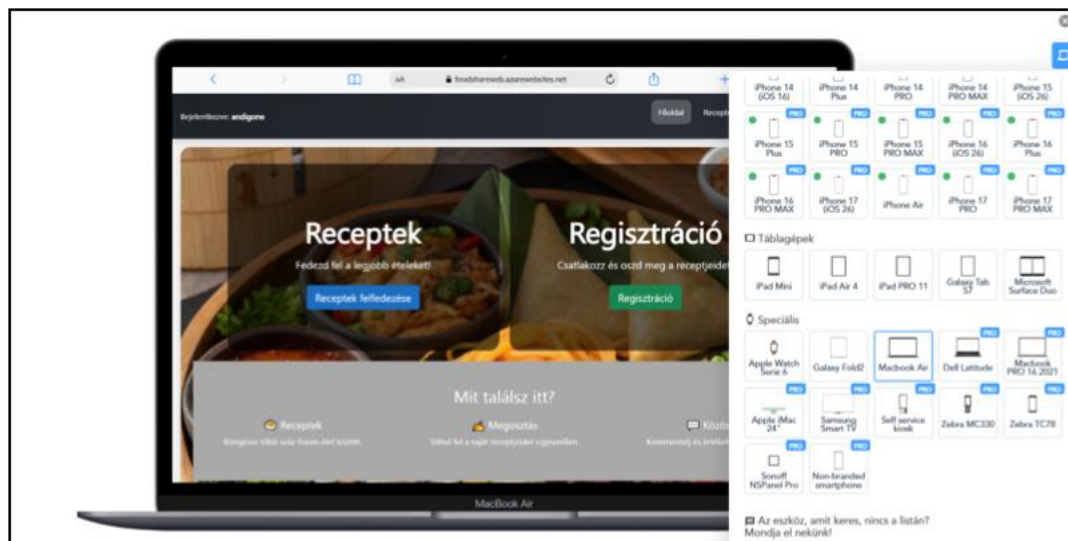
140x900 (gyakori közepkategóriás kijelzőméret)

Mobil és tablet teszteléshez pedig a Mobile First (Google Chrome bővítmény)-t használtuk, ahol több android és Apple telefont és táblagépet teszteltünk:

- Samsung Galaxy S20
- Xiaomi Mi 11i
- iPhone X
- iPhone 13 (iOS 16)



- MacbookAir
- iPad Mini



Kompatibilitás tesztelés

Egy weboldal megjelenése inkonzisztens lehet a különböző böngészőkben vagy böngészőverziókban, így teszteltük az alkalmazásunkat ebből a szempontból is. Az alábbi böngészőkkel néztük meg a kialakított felületet:

Google Chrome / Verzió: 146.0.7680.155 (Hivatalos verzió) (64 bites)

Microsoft Edge vállalati verzió

Firefox / 148.0.2 (64 bites)

A kompatibilitási teszt része a különböző operációs rendszereken történő ellenőrzés is. Az alkalmazásunkat az alábbi operációs rendszereken teszteltük:

Windows 10 Enterprise

Windows 10 Pro

Android 14

Teljesítmény tesztelés

Nagy forgalmi terhelést nem tudtuk tesztelni, manuális adatbevitellel a családtagok segítségével inkább csak stabilitás tesztet végeztünk: párhuzamosan futtattunk több eszközről a recept keresés funkciót, próbáltuk kideríteni, hogy mindennapos működést szimulálva tapasztalható-e bármilyen működési hiba. Ennek során egy indexelési hibát sikerült feltárni és javítani.

Felhasználói dokumentáció

1. A program bemutatása

A Foodshare egy webes alkalmazás, amelynek célja, hogy a felhasználók könnyen megoszthassák egymással kedvenc receptjeiket. A rendszer lehetővé teszi új receptek feltöltését, meglévő receptek böngészését, valamint különböző kategóriák szerinti keresést.

Az alkalmazás egy egyszerűen használható felületet biztosít, ahol a felhasználók:

- új recepteket tölthetnek fel
- böngészhetnek más felhasználók receptjei között
- megtekinthetik a receptek hozzávalóit és elkészítési módját
- rendszerezhetik az ételeket különböző kategóriák szerint

A program célja egy közösségi receptgyűjtő platform létrehozása, amely segíti az embereket az új ételek felfedezésében és a főzési ötletek megosztásában.

2. Hardverkövetelmények

A program egy webes alkalmazás, ezért különleges hardverkövetelményekkel nem rendelkezik. A használatához elegendő egy alapvető számítógép vagy mobil eszköz.

Minimum követelmények

Számítógép vagy laptop:

- Processzor: minimum 1 GHz
- Memória: minimum 2 GB RAM
- Tárhely: legalább 200 MB szabad hely
- Képernyő: minimum 1280×720 felbontás

Mobil eszköz esetén:

- Android vagy iOS okostelefon
- Internetkapcsolat
- Modern böngésző

3. Szoftverkövetelmények

A rendszer használatához az alábbi szoftverek szükségesek:

Böngészők

Az alkalmazás az alábbi böngészőkben működik megfelelően:

- Google Chrome
- Mozilla Firefox
- Microsoft Edge
- Safari
- Brave
- Opera

Ajánlott a böngészők legfrissebb verzióját használni.

Internetkapcsolat

A rendszer használatához folyamatos internetkapcsolat szükséges.

4. A program letöltése, telepítése és indítása

Mivel a Foodshare egy webes alkalmazás, klasszikus értelemben vett telepítésre nincs szükség.

Használat lépései

1. Nyissa meg az internetböngészőt.
2. Adja meg az alkalmazás webcímét.
3. A kezdőoldal automatikusan betöltődik.
4. A felhasználó azonnal elkezdheti a receptek böngészését.

Amennyiben fejlesztői környezetben szeretnénk futtatni:

1. A projekt letöltése GitHubról
2. A fájlok megnyitása Visual Studio programban
3. A projekt futtatása helyi szerveren
4. A böngésző automatikusan megnyitja az oldalt

5. A program használata

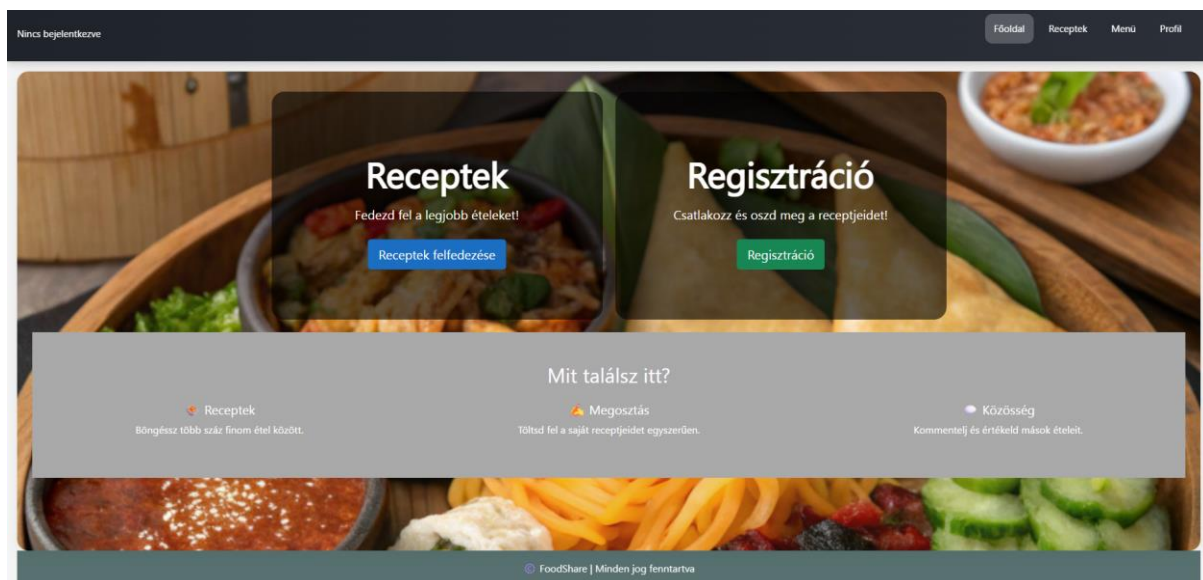
5.1 Főoldal

A főoldalon a felhasználó áttekintheti az elérhető recepteket. Itt találhatóak a legfrissebb vagy legnépszerűbb receptek.

A főoldal tartalmazza:

- navigációs menü
- receptlista
- keresési lehetőség

1. ábra – A kezdőoldal felülete



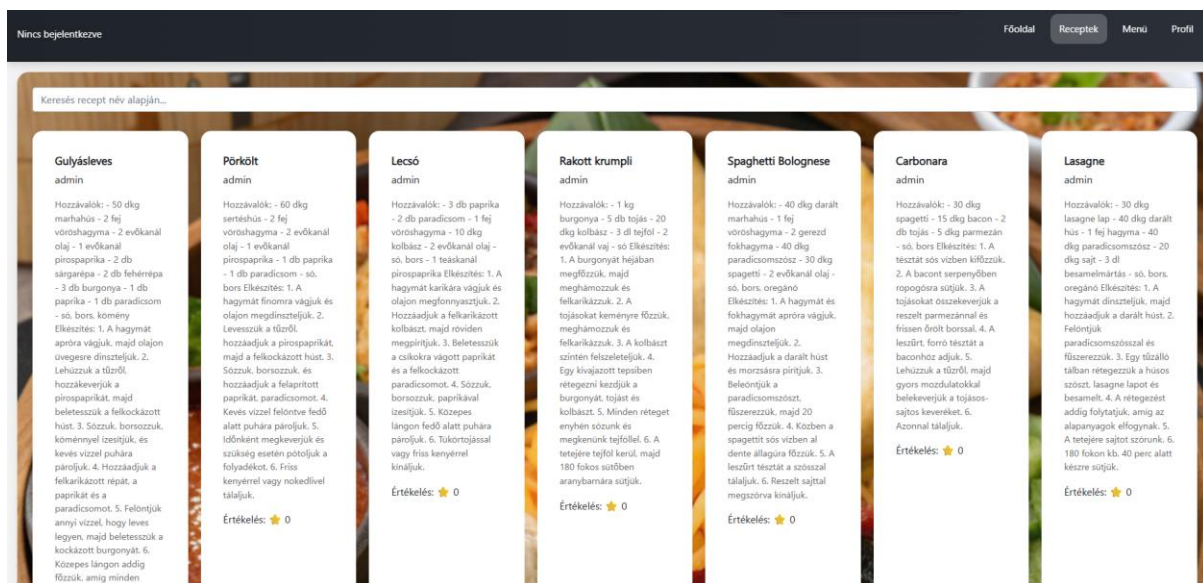
5.2 Receptek böngészése

A receptek oldalon a felhasználó megtekintheti az összes feltöltött receptet.

Minden recept tartalmazza:

- a recept nevét
- a hozzávalók listáját
- az elkészítési módot
- a kategóriát

2. ábra – Receptek listája

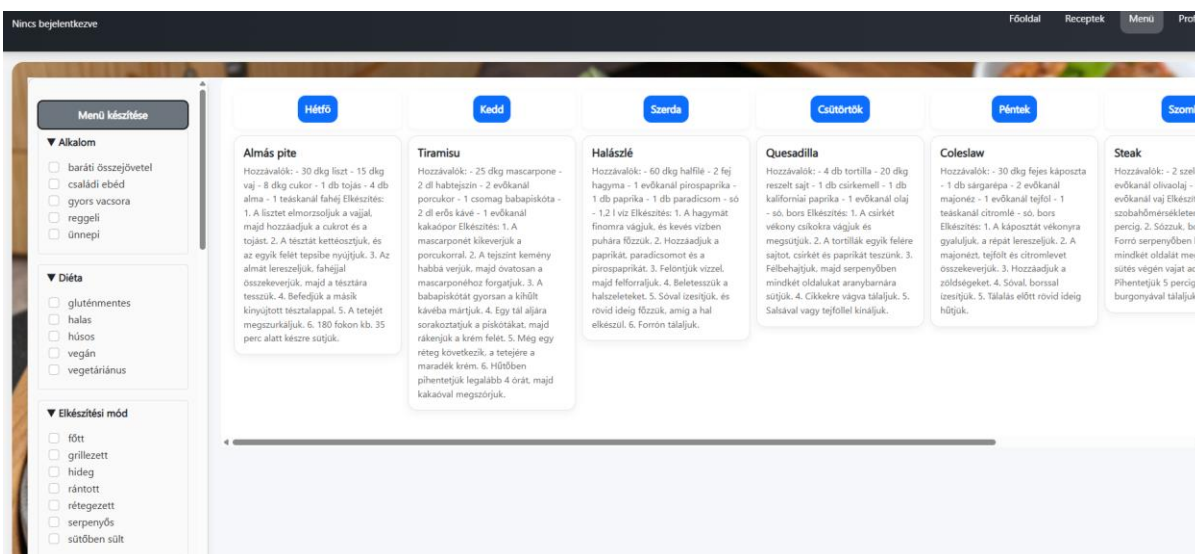


5.3 Menü

A menü oldalon felhasználó létrehozhat egy heti menüt többféle szűrő alapján és azt is megjelölheti, hogy melyik napok legyenek kiválasztva

A menü oldal tartalmazza:

- menü készítése gombot
- hét napjai
- többféle szűrő



5.4 Új recept hozzáadása

A felhasználók új receptet is feltölthetnek az oldalra.

Ehhez egy űrlapot kell kitölteni.

Az űrlap mezői:

Recept neve

- kötelező mező
- maximum 100 karakter
- elfogad ékezetes karaktereket
- kis- és nagybetűk megengedettek

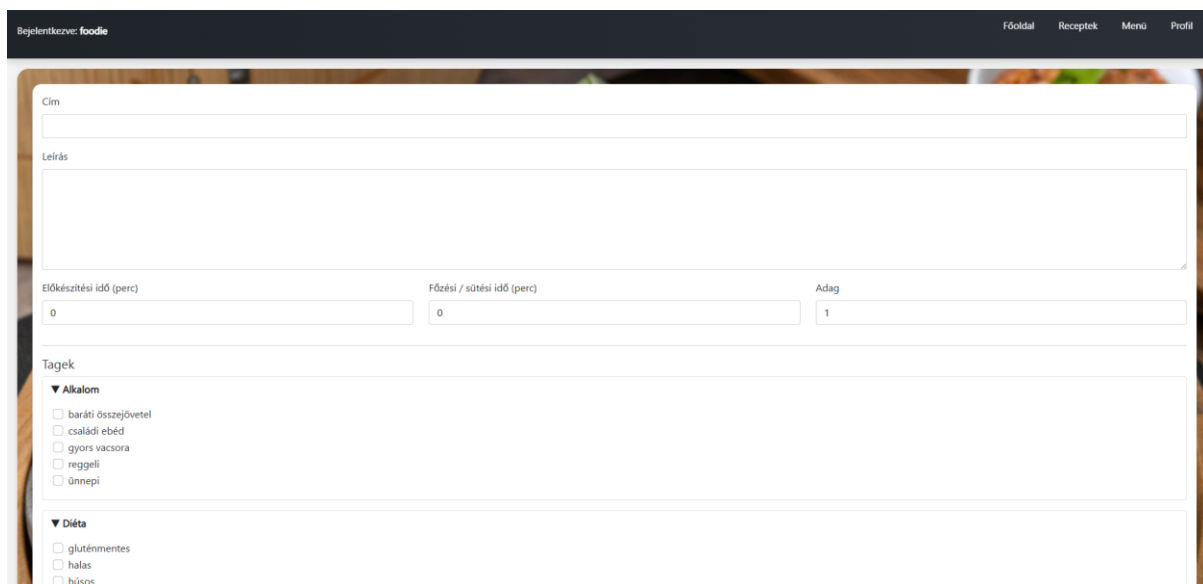
Elkészítés

- kötelező mező
- maximum 2000 karakter
- részletes leírás adható meg

Kategória

- választható lista
- például:
 - leves
 - főétel
 - desszert
 - ital

3. ábra – Recept feltöltő űrlap



The screenshot shows a web application interface for uploading a recipe. At the top, there is a dark navigation bar with the text "Bejelentkezés: foodie" on the left and links for "Főoldal", "Receptek", "Menu", and "Profil" on the right. The main form area is white and contains the following fields and sections:

- Cím**: A text input field for the recipe title.
- Leírás**: A large text area for the recipe description.
- Előkészítési idő (perc)**: A numeric input field with the value "0".
- Főzési / sütési idő (perc)**: A numeric input field with the value "0".
- Adag**: A numeric input field with the value "1".
- Tagok**: A section with two expandable lists:
 - Alkalom**: A list of occasions with checkboxes: "baráti összejövetel", "családi ebéd", "gyors vacsora", "reggeli", and "ünnepi".
 - Diéta**: A list of dietary preferences with checkboxes: "gluténmentes", "halas", and "húsos".

6. Szabadon használható funkciók és regisztrációhoz kötött funkciók

Regisztráció nélkül elérhető funkciók

- receptek böngészése
- recept részleteinek megtekintése
- kategóriák közötti navigáció
- szűrő alapján létrehozható heti menü

Regisztrációhoz kötött funkciók

- új recept feltöltése
- receptek szerkesztése
- receptek törlése
- saját profil kezelése
- komment beszúrása
- recept értékelése

A regisztráció célja, hogy minden feltöltött recepthez tartozzon egy felhasználó.

7. Szövegmezők követelményei

A program több szövegbeviteli mezőt használ. Az alábbi szabályok érvényesek:

Általános szabályok

- ékezetes betűk használata megengedett
- számok használata engedélyezett
- speciális karakterek engedélyezettek

8. Hibajelzések

A program különböző hibajelzéseket adhat.

Hiányzó adat

Ha a felhasználó nem tölt ki egy kötelező mezőt, a rendszer figyelmeztetést jelenít meg.

Példa:

"A cím megadása kötelező."

- A cím megadása kötelező.
- A leírás megadása kötelező.

Túl hosszú szöveg

Ha a felhasználó túllépi a maximális karakterhosszt:

"Hiba: A megadott szöveg túl hosszú."

Hibás bejelentkezés

Ha a felhasználó rossz jelszót ad meg akkor a következő üzenet jelenik meg:

" Sikertelen bejelentkezés.."

Bejelentkezés

Sikertelen bejelentkezés.

9. Admin felhasználó lehetőségei

Az adminisztrátor speciális jogosultságokkal rendelkezik.

Admin funkciók

- receptek szerkesztése
- receptek törlése

Az admin felület segít a rendszer karbantartásában és a tartalom ellenőrzésében.

10. Kapcsolat és támogatás

Amennyiben a felhasználó kérdést szeretne feltenni vagy hibát szeretne jelezni, az alábbi módokon teheti meg.

<https://github.com/Takos05/ReceptMegosztoRepo>

Összefoglalás

A munka értékelése

A projekt elkészítése során sikerült létrehozni egy működőképes receptmegosztó webalkalmazást. A fejlesztés során több fontos informatikai területet is alkalmaztunk, például:

- webfejlesztés
- felhasználói felület tervezése
- adatbáziskezelés
- projektstruktúra kialakítása

A projekt egyik legnagyobb előnye az egyszerű használhatóság. A felhasználók gyorsan eligazodhatnak a felületen, és könnyedén feltölthetik saját receptjeiket.

A fejlesztés során külön figyelmet fordítottunk arra, hogy az oldal áttekinthető és logikus felépítésű legyen.

A program jövőbeli hasznosulása

A Foodshare alkalmazás a jövőben tovább fejleszthető.

Lehetséges fejlesztési irányok:

- kedvenc receptek mentése
- heti menü mentése
- keresőfunkció fejlesztése
- mobilalkalmazás készítése

A program akár egy nagyobb közösségi receptoldal alapja is lehet. A rendszer lehetőséget ad arra, hogy az emberek megosszák egymással főzési tapasztalataikat és új ételeket fedezzenek fel.

Összességében a projekt sikeresen teljesíti a kitűzött célokat, és megfelelő alapot biztosít a további fejlesztésekhez.

Köszönetnyilvánítás

Külön köszönjük Méhes József György tanárúrnak a segítségét az Autentikáció működésében és családuknak a felhasználói élmény tesztelésében.